

Apuntes

Víctor Manuel Peiró Martínez

FIIS 2A

Diseño de Software

Tabla de contenido

Tema 0 – Introducción	3
0.1 – Relaciones	3
0.2 – UML	4
Tema 1 – Patrones de Diseño	4
1.1 – Patrón Strategy	5
Ejemplo:.....	6
1.2 – Patrón Observer	7
Ejemplo.....	8
1.3 – Patrón State	9
Ejemplo.....	10
1.4 – Patrón Singleton	11
Ejemplo:.....	11
1.5 – Patrón Decorator.....	12
Ejemplo:.....	13
1.6 – Patrón Factory Method	14
Ejemplo:.....	14
1.7 – Patrón Abstract Factory Method.....	16
Ejemplo:.....	16
1.8 – Patrón Template.....	18
Ejemplo:.....	18
1.9 – Patrón Adapter	19
Ejemplo.....	20
1.10 – Patrón Facade	22
Ejemplo.....	22
Tema 2 – Refactoring.....	24

Tema 0 – Introducción

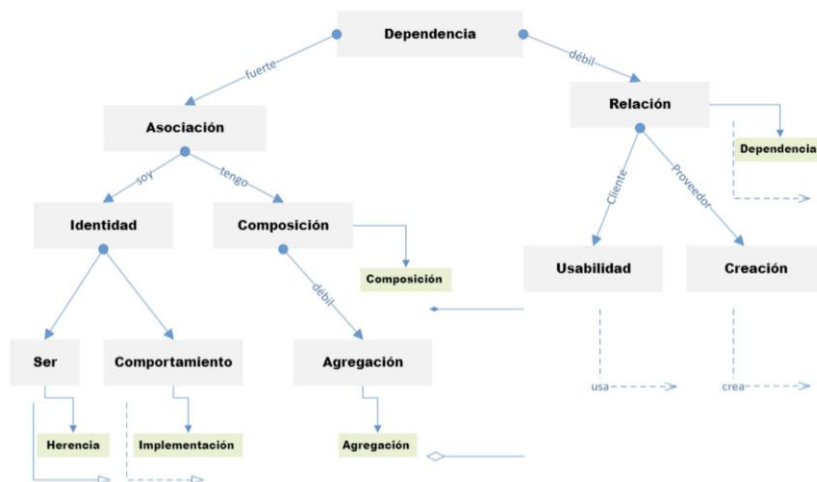
0.1 – Relaciones

Relaciones de Asociación:

- **Composición:** Relaciones del tipo tiene un. Ejemplo: Un coche tiene un motor
 - **Composición Dura:** La vida de la clase contenida coincide con la vida de la clase contenedora. Es decir, si eliminamos el contenedor, se elimina el componente.
 - Los componentes no pueden ser compartidos por varios objetos
 - **Agregación (Composición Blanda):** La vida de la clase contenida no coincide con la vida de la clase contenedora. Es decir, si eliminamos el contenedor, no se elimina el componente.
 - Los componentes pueden ser compartidos por varios objetos
- **Identidad:** Relaciones de tipo es un. El hijo solo es una instancia concreta. Ejemplo, un perro es un animal
 - **Especialización:** Herencia. Se especializa un metodo
 - **Comportamiento:** Implementación

Relaciones de Relación:

- **Usabilidad:** Un objeto usa otro
- **Creación:** Un objeto crea otro



Delegación: Dos objetos se deben encargar de una petición, pero solo se define en uno de ellos.

- Delegación por composición
- Delegación por inyección

0.2 – UML

UML: Unified Modeling Language

Los atributos se definen como: **visibilidad nombre : tipo = valor inicial**(si existe)

Ej: - nombre: String = “victor”

Los métodos se definen como: **visibilidad nombre(tipoAtributo1, tipoAtributo2,...) : tipo devuelto**

Ej: + calcularArea(Double) : Double

Visibilidad:

- -: Privado
- +: Publico
- #: Protegido
- ~: Por defecto

En cursiva o entre <<>>: Interfaces y Metodos/Clases Abstractas

Subrayado: Métodos/Clases Estáticas

Relaciones:

- **Herencia:** Flecha con linea recta
- **Implementación:** Flecha con line en trocitos - - >
- **Composición:** Rombo relleno
- **Agregación:** Rombo vacio

Tema 1 – Patrones de Diseño

	De Creación	Estructurales	Comportamiento
De Clase	Factory Method	Class Adapter	Template Method
De Objeto	Abstract Factory	Decorator	Observer
	Singleton	Facade	State
		Object Adapter	Strategy

De Clase: Se ocupan de relaciones entre clases y sus especializaciones a través de la herencia

De Objeto: Tratan relaciones sobre objetos

Creacionales: Tiene que ver con el proceso de creación entre objetos

Estructurales: Tratan la composición entre clases y objetos

Comportamiento: Caracterizan la forma en las que interactúan los objetos

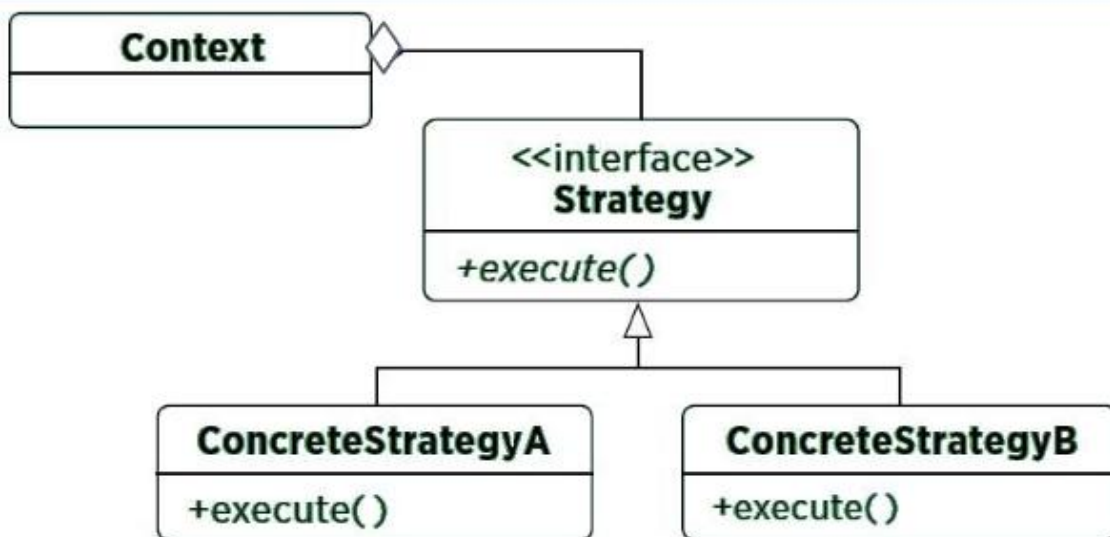
1.1 – Patrón Strategy

Definición: Describe como implementar una familia de algoritmos encapsulándolos para hacerlos intercambiables.

Basado en la composición, distintas maneras de hacer lo mismo.

Partes:

- **Estrategia:** Define una interfaz común a los algoritmos que soporta
- **Estrategias Concretas:** Define una implementación concreta con mediante la interfaz.
- **Contexto:** Está compuesto por un objeto de tipo Estrategia e instanciado con una estrategia concreta. Va por composición blanda.



Ejemplo:

Contexto:

```
public class GuarderiaStrategyContext implements MedicalStrategy {  
    private MedicalStrategy medicalStrategy; //Relacion de Composicion Blanda  
  
    public GuarderiaStrategyContext() {  
        this(new OculistStrategy());  
    }  
  
    public GuarderiaStrategyContext (MedicalStrategy medicalStrategy) {  
        this.medicalStrategy = medicalStrategy;  
    }  
  
    public MedicalStrategy getMedicalStrategy() {  
        return this.medicalStrategy;  
    }  
  
    public void setMedicalStrategy(MedicalStrategy medicalStrategy) {  
        this.medicalStrategy = medicalStrategy;  
    }  
  
    @Override  
    public void applyMedicalStrategy() {  
        this.medicalStrategy.applyMedicalStrategy(); //Delegacion por agregacion  
    }  
}
```

Estrategia/Superclase común:

```
public interface DoctorStrategy extends MedicalStrategy{  
    public void examineKids();  
    public void sendBills();  
    public void sendResults();  
}
```

```
public interface MedicalStrategy{  
    public void applyMedicalStrategy();  
}
```

Estrategia Concreta

```
public class LogopedaStrategy implements DoctorStrategy {  
    private String name;  
  
    public LogopedaStrategy() {  
        this("Dr. Fong");  
    }  
  
    public LogopedaStrategy(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void examineKids() {  
        System.out.println("[Logopeda] " + this.name + " examinando a los niños.");  
    }  
  
    @Override  
    public void sendBills() {  
        System.out.println("[Logopeda] " + this.name + " enviando facturas.");  
    }  
  
    @Override  
    public void sendResults() {  
        System.out.println("[Logopeda] " + this.name + " enviando resultados.");  
    }  
  
    @Override  
    public void applyMedicalStrategy() {  
        this.examineKids();  
        this.sendBills();  
        this.sendResults();  
    }  
}
```

1.2 – Patrón Observer

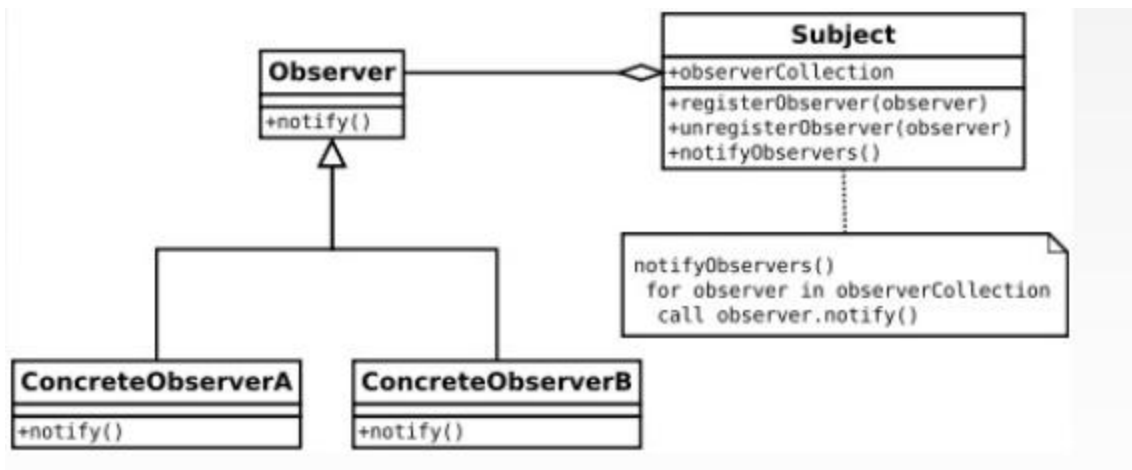
Definición: Define una dependencia de uno a muchos, de forma de que cuando uno cambie de estado, se notifique a todo objeto que dependa de él.

Partes:

- **Observador:** Objeto que necesita ser informado de los cambios de estado
- **Observable:** Objeto que es monitorizado

Modelos:

- **Push:** El observable se envía a todos los observadores con todo lo relacionado con el cambio de estado. Se pasa directamente el nuevo estado así que no se necesita saber sobre el modelo completo
- **Pull:** El observable solo informade que hay un cambio de estado y el observador inserado le saca la información.
- **PullPush:** Combinación de ambos



Ejemplo

Observador:

```
public interface PullPushObserver {
    public void update(PullPushObservable observable, Object object);
}
```

```
public class BinaryObserver implements PullPushObserver {
    @Override
    public void update(PullPushObservable observable, Object object) {
        if (observable instanceof ObservableNumber) { //Protocol Pull
            System.out.println("[Pull Protocol] Binary Integer as String: " + Integer.toBinaryString(((ObservableNumber)observable).getNumber()));
        }
        if (object instanceof Integer) { //Protocol Push
            System.out.println("[Push Protocol] Binary Integer as String: " + Integer.toBinaryString((Integer)object));
        }
    }
}
```

Observable:

```
public interface PullPushObservable {
    public void attach(PullPushObserver observer);
    public void detach(PullPushObserver observer);
    public void notifyObservers();
}
```

```

public class ObservableNumber implements PullPushObservable {

    private List<PullPushObserver> observers; //El observable guarda los observadores a los que notificar
    private Integer number; //Estado a observar

    public ObservableNumber(Integer number) {
        this(number, new ArrayList<PullPushObserver>());
    }

    public ObservableNumber(Integer number, List<PullPushObserver> observers) {
        this.number = number;
        this.observers = observers;
    }

    public Integer getNumber() {
        return this.number;
    }

    public void setNumber(Integer number) { //Se notifica del cambio de estado con el setter
        this.number = number;
        this.notifyObservers();
    }

    @Override
    public void attach(PullPushObserver observer) { //Se añade un observador a la lista
        this.observers.add(observer);
    }

    @Override
    public void detach(PullPushObserver observer) { //Se elimina un observador de la lista
        this.observers.remove(observer);
    }

    @Override
    public void notifyObservers() { //Se informa a todos los observadores
        for (PullPushObserver observer : this.observers) {
            observer.update(this, this.number);
        }
    }

}

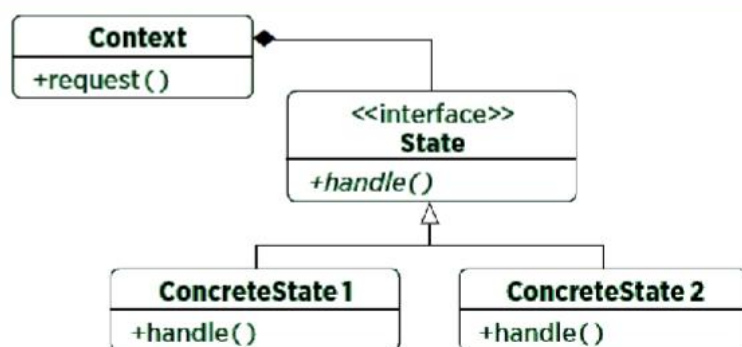
```

1.3 – Patrón State

Definición: Permite que un objeto modifique su estado cada vez que cambie su estado interno

Partes:

- **Contexto:** Proporciona una interfaz al cliente para realizar alguna acción y delega solicitudes específicas de estado.
- **Estado/Superclase Común:** Encapsula el comportamiento asociado con un estado particular del contexto
- **Estados Concretos:** Clases concretas que implementan un estado



Ejemplo

Contexto

```
public class TrafficLight {

    private TrafficLightState trafficLightState;
    private RedLightState redLight;
    private AmberLightState amberLight;
    private GreenLightState greenLight;

    public TrafficLight() {
        this.redLight = new RedLightState(this);
        this.amberLight = new AmberLightState(this);
        this.greenLight = new GreenLightState(this);
        this.setTrafficLightState(this.redLight);
    }

    public TrafficLightState getTrafficLightState() {
        return this.trafficLightState;
    }

    public void setTrafficLightState(TrafficLightState trafficLightState) {
        this.trafficLightState = trafficLightState;
    }

    public RedLightState getRedLight() {
        return this.redLight;
    }

    public AmberLightState getAmberLight() {
        return this.amberLight;
    }

    public GreenLightState getGreenLight() {
        return this.greenLight;
    }

    public void showState() {
        this.trafficLightState.showState(); //Delegacion por composicion
    }
}
```

Estado:

```
public interface TrafficLightState {
    public void showState();
}
```

Estado Concreto

```
public class RedLightState implements TrafficLightState{

    private TrafficLight trafficLight; //Se le pasa el contexto al estado concreto para manejar los estados internos

    public RedLightState (TrafficLight trafficLight) {
        this.trafficLight = trafficLight;
    }

    @Override
    public void showState() {
        System.out.println("Red Light!!!");
        this.trafficLight.setTrafficLightState(this.trafficLight.getGreenLight());
    }

}
```

1.4 – Patrón Singleton

Definición: Garantiza que una clase tenga una sola instancia y proporciona un acceso global a ella



Como Detectar:

- Constructor Privado
- Metodo getInstance

Ejemplo:

Instanciación Perezosa

```
public class SoupOnDemand {  
    public static SoupOnDemand soupOnDemand;  
  
    private SoupOnDemand() {}  
  
    public static SoupOnDemand getInstance() {  
        if (Objects.isNull(soupOnDemand)) {  
            soupOnDemand = new SoupOnDemand();  
        }  
        return soupOnDemand;  
    }  
  
    public void ready() {  
        System.out.println("Soup On Demand is ready");  
    }  
}
```

Instanciación Normal

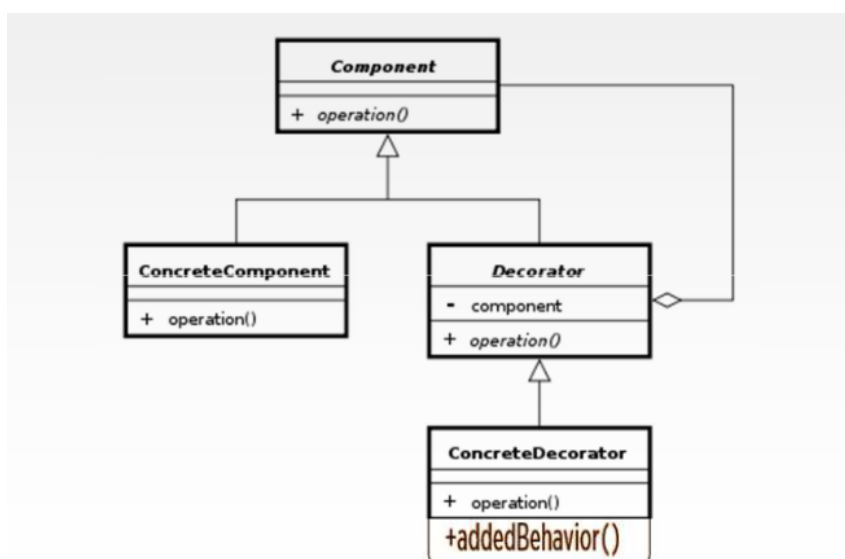
```
public class SingletonSoup {  
  
    private static SingletonSoup singletonSoup = new SingletonSoup();  
  
    private SingletonSoup() {}  
  
    public static SingletonSoup getInstance() {  
        return singletonSoup;  
    }  
  
    public void ready() {  
        System.out.println("Singleton Soup is ready");  
    }  
  
}
```

1.5 – Patrón Decorator

Definición: Añade dinámicamente nuevas responsabilidades a un objeto proporcionando una alternativa flexible a la herencia. Permite agregar dinámicamente funcionalidades a un objeto individual.

Partes:

- **Component:** Define la interfaz a la que se le pueden asignar nuevas responsabilidades
- **Decorator:** Clase Abstracta que mantiene una referencia al objeto Componente de define una interfaz uniforme
- **DecoratorConcreto:** Añade funcionalidades a componente
- **Componente Concreto:** La clase que esta siendo decorada



Ejemplo:

Componente:

```
public interface PrizeComponent {  
    public String getDescription();  
}
```

Componente Base/Concreto:

```
public class StickerPrizeBaseComponent implements PrizeComponent{  
    private String description;  
  
    public StickerPrizeBaseComponent() {  
        this("");  
    }  
  
    public StickerPrizeBaseComponent(String description) {  
        this.description = description;  
    }  
  
    @Override  
    public String getDescription() {  
        return this.description;  
    }  
}
```

Decorador:

```
public abstract class PrizeComponentDecorator implements PrizeComponent{  
    private PrizeComponent prize;  
  
    public PrizeComponentDecorator(PrizeComponent prize) {  
        this.prize = prize;  
    }  
  
    @Override  
    public String getDescription() {  
        String description = "";  
  
        if (this.prize != null) {  
            description = this.prize.getDescription();  
        }  
  
        description += " " + this.getPrize();  
        return description;  
    }  
  
    public abstract String getPrize();  
}
```

Decorador Concreto:

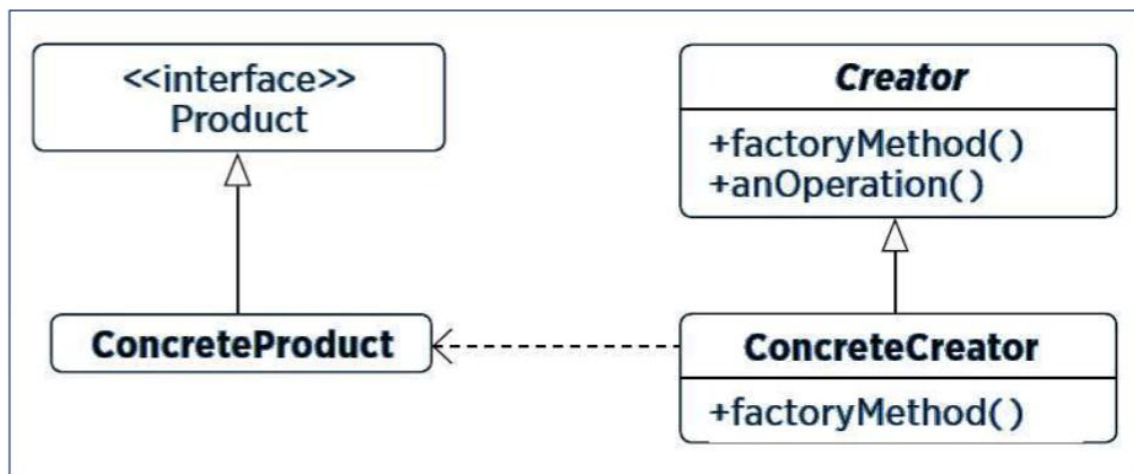
```
public class CertificateComponentDecorator extends PrizeComponentDecorator {  
    public CertificateComponentDecorator(PrizeComponent prize) {  
        super(prize);  
    }  
  
    @Override  
    public String getPrize() {  
        return "con certificado por ir al baño";  
    }  
}
```

1.6 – Patrón Factory Method

Definición: Define una interfaz para crear un objeto, pero deja que las subclases quienes decidan que clase instanciar

Partes:

- **Product:** Una representación abstracta del producto
- **Concrete Product:** Un producto
- **Creator/Factory:** Una representación abstracta del recurso creador centralizada
- **Concrete Creator:** Una fábrica concreta



Ejemplo:

Producto

```
public interface Animal {  
    public Integer getNumLegs();  
    public void makeNoise();  
}
```

Producto Concreto

```
public class Dog implements Animal {  
    @Override  
    public Integer getNumLegs() {  
        return 4;  
    }  
  
    @Override  
    public void makeNoise() {  
        System.out.println("Woof");  
    }  
}
```

Creator:

```
public interface AnimalFactory {  
    public Animal createAnimal();  
}
```

Creator Concreto

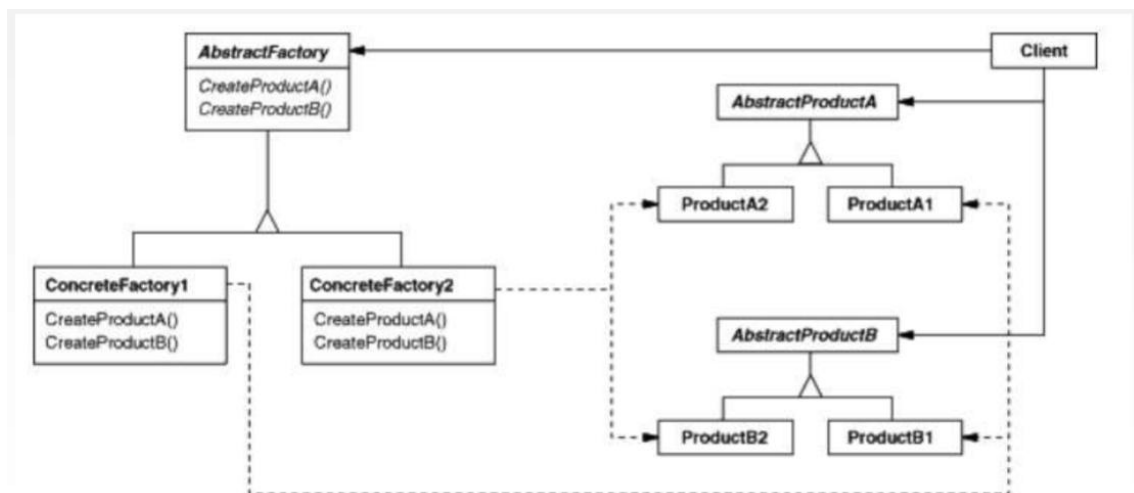
```
public class RandomAnimalFactory implements AnimalFactory{  
  
    public static Double getRandomBetweenRange(Integer min, Integer max) {  
        return (Math.random()*(max-min))+min;  
    }  
  
    @Override  
    public Animal createAnimal() {  
        Animal animal;  
        Double randomNum = RandomAnimalFactory.getRandomBetweenRange(0, 100);  
  
        if (randomNum >= 50) {  
            animal = new Dog();  
        } else {  
            animal = new Cat();  
        }  
  
        return animal;  
    }  
}
```

1.7 – Patrón Abstract Factory Method

Definición: Proporciona una interfaz para crear familias de objetos que dependen entre sí.

Partes:

- **Cliente:** Clase que llama a la factoria adecuada ya que necesita crear uno de los objetos que provee la factoria
- **FactoriaAbstracta:** Definición de las interfaces de las factorías. Debe proveer un método de cada objeto que pueda crear
- **FactoriasConcretas:** Diferentes Familias de productos
- **ProductoAbstracto:** Definicion de las interfaces para la familia de productos genericos
- **ProductoConcreto:** Implementacion de los diferentes productos



Ejemplo:

Producto Abstracto:

```
public interface Daemon extends Enemy{
    public String getColor();
}
```

Factoria Abstracta:

```
public interface AbstractEnemyFactory {
    public Enemy createDaemon();
    public Enemy createWitch();
}
```

Producto Concreto:

```
public class W1Daemon implements Daemon{

    private String name;
    private Integer power;
    private Integer lives;
    private String color;

    public W1Daemon() {
        this.name = "Daemon";
        this.power = 10 * World.World1.getComplexFactor();
        this.lives = 1 * World.World1.getComplexFactor();
        this.color = "Rojo";
    }

    @Override
    public String getName() {
        return this.name;
    }

    @Override
    public Integer getPower() {
        return this.power;
    }

    @Override
    public Integer getLives() {
        return this.lives;
    }

    @Override
    public String getColor() {
        return this.color;
    }

    @Override
    public String toString() {
        return "W1Daemon [name=" + name + ", power=" + power + ", lives=" + lives + ", color=" + color + "]";
    }

}
```

Factoria Concreta:

```
public class W1Factory implements AbstractEnemyFactory{

    @Override
    public Enemy createDaemon() {
        return new W1Daemon();
    }

    @Override
    public Enemy createWitch() {
        return new W1Witch();
    }

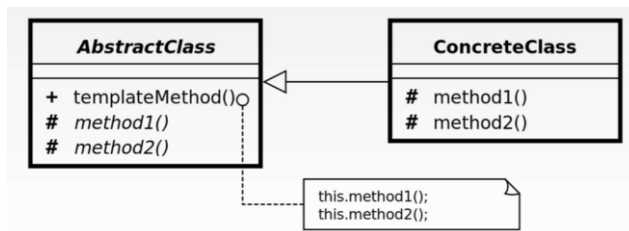
}
```

1.8 – Patrón Template

Definición: Define en una operación el esqueleto de un algoritmo, delegando en subclases algunos de sus casos

Partes:

- **Plantilla:** Clase Abstracta que actúa como plantilla
- **Clase Concreta:** Clase sobre la que se aplica la plantilla



Ejemplo:

Plantilla:

```
//Template para las estrategias concretas
public abstract class AbstractDoctorStrategy implements DoctorStrategy{

    protected String name;

    public AbstractDoctorStrategy(String name) {
        this.name = name;
    }

    @Override
    public void applyMedicalStrategy() {
        this.examineKids();
        this.sendResults();
        this.sendBills();
    }

    public abstract void examineKids();
    public abstract void sendBills();
    public abstract void sendResults();
}
```

Clase Concreta:

```
public class OculistStrategy extends AbstractDoctorStrategy{

    public OculistStrategy() {
        this("Dr. Wang");
    }

    public OculistStrategy(String name) {
        super(name);
    }

    @Override
    public void examineKids() {
        System.out.println("[Oculista] " + this.name + " examinando a los niños.");
    }

    @Override
    public void sendBills() {
        System.out.println("[Oculista] " + this.name + " enviando facturas.");
    }

    @Override
    public void sendResults() {
        System.out.println("[Oculista] " + this.name + " enviando resultados.");
    }

}
```

1.9 – Patrón Adapter

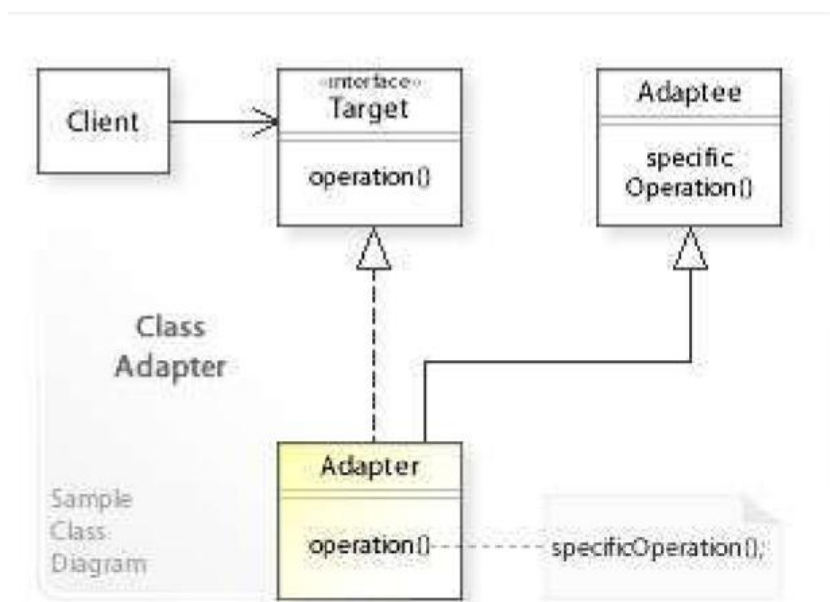
Definición: Convierte una interfaz en otra distinta que la que esperan los clientes. Permite que cooperen clases que de otra manera no podrían.

Partes:

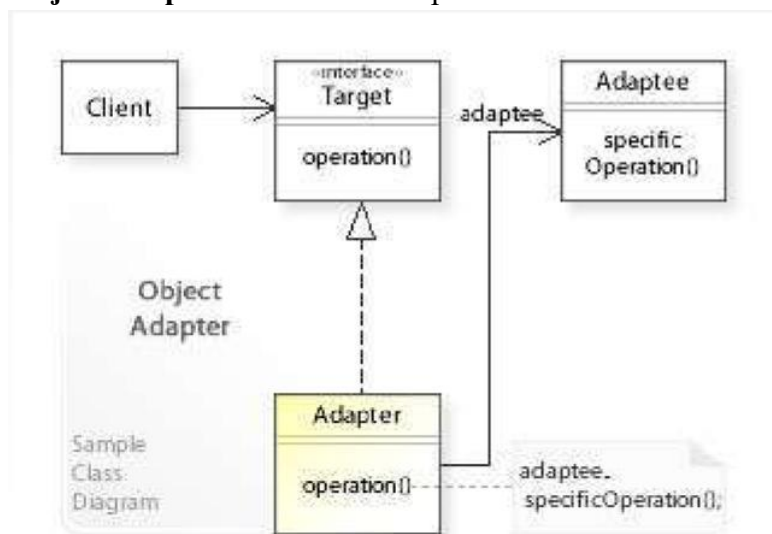
- **Adapter/Target:** Define la interfaz que utiliza el cliente
- **ConcreteAdapter:** Adapta el interfaz del objeto adaptado
- **Adaptee:** Define una interfaz existente que necesita adaptarse
- **Client:** Clase que colabora con los objetos que implementan la interfaz Adapter

Diseños:

- **Class Adapter:** Mediante Herencia



- **Object Adapter:** Mediante Composicion



Ejemplo

Adaptee

```
//Adaptee --> Se adapta el voltaje que recibe
public class EnergySocket {

    private Volt volt;

    public EnergySocket() {
        this(new Volt(120));
    }

    public EnergySocket(Volt volt) {
        this.volt = volt;
    }

    public Integer getVolts() {
        return this.volt.getVolts(); //Delegacion por agregacion
    }

    public Volt getVolt() {
        return this.volt;
    }

    public void setVolt(Volt volt) {
        this.volt = volt;
    }
}
```

Adapter/Target

```
//Adapter/Target --> Los metodos que vamos a adaptar
public interface EnergySocketAdapter extends EnergySocketService{
    public Volt get3Volt();
    public Volt get12Volt();
    public Volt get120Volt();
}
```

Client

```
//Cliente
public interface EnergySocketService {
    public Volt getVolt();
}
```

Class Adapter

```
//Adaptador Concreto --> Lo que va a adaptar el objeto adaptado
public class EnergySocketClassAdapter extends EnergySocket implements EnergySocketAdapter{

    @Override
    public Volt getVolt() {
        return super.getVolt();
    }

    @Override
    public Volt get3Volt() {
        return this.convertVolts(this.getVolt(), 40);
    }

    @Override
    public Volt get12Volt() {
        return this.convertVolts(this.getVolt(), 10);
    }

    @Override
    public Volt get120Volt() {
        return this.convertVolts(this.getVolt(), 1);
    }

    public Volt convertVolts(Volt v, Integer i) {
        return new Volt(v.getVolts()/i);
    }

}
```

Object Adapter

```
public class EnergySocketObjectAdapter implements EnergySocketAdapter{

    private EnergySocket energySocket;

    public EnergySocketObjectAdapter(EnergySocket energySocket) {
        this.energySocket = energySocket;
    }

    @Override
    public Volt getVolt() {
        return this.energySocket.getVolt(); //Delegacion por agregacion
    }

    @Override
    public Volt get3Volt() {
        return this.convertVolts(40);
    }

    @Override
    public Volt get12Volt() {
        return this.convertVolts(10);
    }

    @Override
    public Volt get120Volt() {
        return this.getVolt();
    }

    public Volt convertVolts(Integer i) {
        return new Volt(this.energySocket.getVolts() / i);
    }

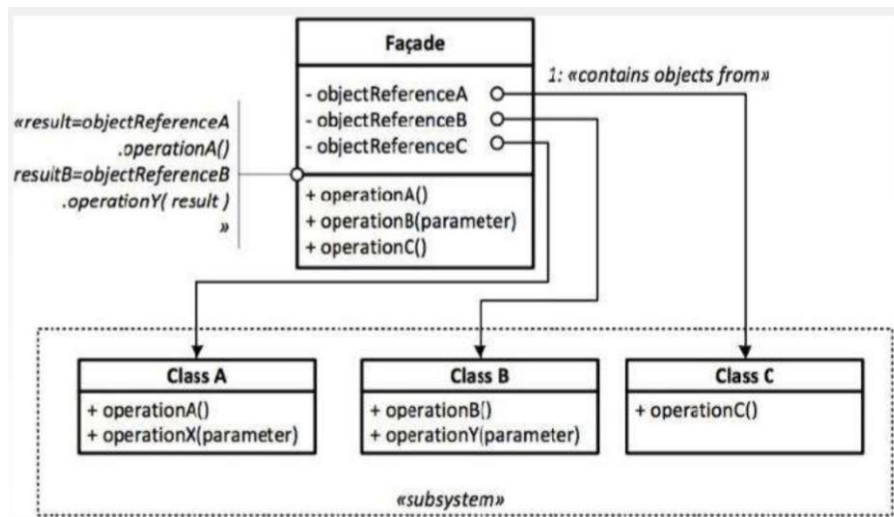
}
```

1.10 – Patrón Facade

Definición: Proporciona una interfaz unificada para un conjunto de interfaces de un subsistema

Partes:

- **Facade:** Sabe que clases del subsistema son responsables de una solicitud. Delega las solicitudes a los objetos adecuados
- **Subsystem Classes:** Implementa la funcionalidad propia del subsistema



Ejemplo

Subsistema:

```
public class ScheduleServer {

    public void startBooting() {
        System.out.println("Starts Booting...");
    }

    public void readSystemConfigFiles() {
        System.out.println("Reading system config files...");
    }

    public void init() {
        System.out.println("Initializing...");
    }

    public void initContext() {
        System.out.println("Initializing Context...");
    }

    public void initListeners() {
        System.out.println("Initializing Listeners...");
    }

    public void createSystemObjects() {
        System.out.println("Creating system objects...");
    }

    public void releaseProcesses() {
        System.out.println("Realising processes...");
    }

    public void destroy() {
        System.out.println("Destroy...");
    }

}
```

Facade:

```
public class ScheduleServerFacade {  
    private ScheduleServer server;  
  
    public ScheduleServerFacade(ScheduleServer scheduleServer) {  
        this.server = scheduleServer;  
    }  
  
    public void startServer() {  
        server.startBooting();  
        server.readSystemConfigFiles();  
        server.init();  
        server.initContext();  
        server.initListeners();  
        server.createSystemObjects();  
    }  
  
    public void stopServer() {  
        server.releaseProcesses();  
        server.destroy();  
        server.destroySystemObjects();  
        server.destroyListeners();  
        server.destroyContext();  
        server.shutdown();  
    }  
}
```

Tema 2 – Refactoring

Código Limpio:

- Pasa todas las pruebas
- No contiene duplicados
- Muestra claramente la intención del desarrollador
- Minimiza el numero de clases y métodos para expresar dicha intención

Refactorización: Proceso de tomar el código base y mejorarlo en código limpio

Cuando hacer Refactoring:

- Durante la revisión del código por la presencia de code smells
- Se ha encontrado un bug
- Se quieren añadir nuevas funcionalidades

Code Smells:

1. **Código duplicado:** Mismo código en varios lugares. Mejor unificarlo.
2. **Método largo:** Métodos extensos son difíciles de entender. Mejor dividirlos.
3. **Clase grande:** Hace demasiadas cosas. Debería dividirse.
4. **Lista de parámetros larga:** Muchos argumentos indican falta de encapsulamiento.
5. **Cambio divergente:** La clase cambia por diferentes razones. Mejor separarla.
6. **Cura de un escopetazo:** Un cambio requiere tocar muchas clases dispersas. Deberían agruparse.
7. **Envidia de capacidades:** Un método usa más datos de otra clase que de la suya. Debería moverse.
8. **Agrupaciones de datos:** Datos que siempre van juntos deberían estar en una clase.
9. **Obsesión por tipos primitivos:** Usar primitivos en lugar de clases específicas reduce claridad.
10. **Sentencias switch:** Muchos *switch* similares dificultan el mantenimiento. Usar polimorfismo.
11. **Clase vaga:** Clase que no hace nada útil. Eliminarla.
12. **Jerarquías paralelas:** Añadir una clase obliga a crear otra en paralelo. Mal diseño.
13. **Generalización especulativa:** Código preparado para casos que no se necesitan. Eliminarlo.

- 14. **Campo temporal:** Variables de instancia que se usan sólo en ciertos casos. Confunden.
- 15. **Cadenas de mensajes:** Muchas llamadas encadenadas entre objetos. Acopla en exceso.
- 16. **Hombre en el medio:** Clase que solo delega llamadas. Puede eliminarse.
- 17. **Intimidad inapropiada:** Clases que acceden a detalles internos de otras. Desacoplar.
- 18. **Interfaces diferentes:** Clases que hacen lo mismo pero con interfaces distintas. Unificar.
- 19. **Clase de librería incompleta:** Falta funcionalidad en librerías externas. Añadir adaptadores.
- 20. **Clase de datos:** Sólo tiene atributos y getters/setters. Debe tener comportamiento.
- 21. **Legado rechazado:** Subclase que no usa lo heredado. Mal uso de la herencia.
- 22. **Comentarios:** Si hay muchos comentarios, probablemente el código sea confuso. Mejor refactorizar.

2.1 - Técnicas Refactoring

Composing Methods:

- 1. **Extract Method**
Mueve fragmentos de código a un nuevo método con un nombre descriptivo.
- 2. **Inline Method**
Sustituye llamadas a un método por su contenido si el método es innecesario.
- 3. **Extract Variable**
Descompón expresiones complejas en variables con nombres claros.
- 4. **Inline Temp**
Reemplaza variables temporales que sólo almacenan una expresión simple.
- 5. **Replace Temp with Query**
Convierte una variable temporal en un método que devuelva su valor.
- 6. **Split Temporary Variable**
Usa una variable diferente para cada propósito si una variable temporal tiene múltiples usos.
- 7. **Remove Assignments to Parameters**
Evita modificar parámetros directamente, usa variables locales.

8. Replace Method with Method Object

Convierte un método largo en una clase para facilitar su descomposición.

9. Substitute Algorithm

Reemplaza un algoritmo por uno más claro o eficiente.

Moving Features Between Objects

10. Move Method

Traslada un método a la clase que más lo usa.

11. Move Field

Traslada un atributo a la clase donde tiene más sentido.

12. Extract Class

Divide una clase en dos si tiene demasiadas responsabilidades.

13. Inline Class

Fusiona una clase sin funcionalidad real con otra.

14. Hide Delegate

Encapsula llamadas a objetos delegados para evitar dependencias externas.

15. Remove Middle Man

Elimina métodos que solo delegan a otros objetos.

16. Introduce Foreign Method

Agrega un método a una clase cliente cuando no se puede modificar la clase objetivo.

17. Introduce Local Extension

Extiende una clase existente mediante una subclase o clase auxiliar si no puedes modificarla.

Organizing Data

18. Self Encapsulate Field

Accede a campos mediante getters y setters, incluso internamente.

19. Replace Data Value with Object

Convierte un valor simple en un objeto si necesita lógica adicional.

20. Change Value to Reference

Sustituye múltiples objetos idénticos por una única referencia compartida.

21. Change Reference to Value

Convierte referencias poco modificadas en valores independientes.

22. Replace Array with Object

Sustituye arreglos con estructura fija por objetos con atributos explícitos.

23. Duplicate Observed Data

Separa datos de dominio y de interfaz en objetos distintos y sincronizados.

24. Change Unidirectional Association to Bidirectional

Haz bidireccional una relación entre clases si ambas se necesitan mutuamente.

25. Change Bidirectional Association to Unidirectional

Elimina una dirección de una relación si ya no es necesaria.